

Test Plan

Unit, Integration, System, and User Acceptance Testing
for the AI-Driven High School Data Intelligence Platform

Max Chalekson

Sheng Hu

Qifan Yang

Northwestern University
Master of Science in Data Science

Prepared for Bob Henkins
Northwestern Undergraduate Admissions and Enrollment

Summer 2026

Contents

Overview	1
1 Test Coverage	1
2 Test Scenarios	3
3 Test Case Design	5
4 Test Environment Setup	6
5 Test Execution Plan	8
6 Defect Management	10
7 Regression Testing	10
8 Documentation and Reporting	11
9 Sample Test Report	11

Overview

This plan covers unit, integration, system, and user acceptance testing for the Northwestern high-school data platform (Sections 3–4 of `written-report-iterations/MSDS_498_-version-wk3.pdf`). It is not a generic template — every scenario below is grounded in a specific, real risk already found in this pipeline, several of them actual bugs fixed during development (see the “why this test exists” notes throughout). A runnable test suite backing this plan lives in `tests/`: **57 automated tests, all passing as of this writing** — 55 unit/integration/CSV-system tests plus 2 that bring up the real Docker/Postgres pipeline end to end (see the Test Execution Plan for how to run them).

1 Test Coverage

Four testing types are used, matched to where risk actually concentrates in this project, as summarized in Table 1.

Table 1: Testing types, matched to where risk concentrates in the pipeline.

Type	What it covers	Why it matters here
Unit	Individual functions in <code>etl/build_features.py</code> , <code>etl/combine_schools.py</code> (its two pure-Python helpers only — see gap below), <code>etl/build_rigor_classification.py</code> , <code>etl/build_modeling_dataset.py</code>	This is where the real bugs have lived: sector misclassification, IB match-tier gating, LEAID derivation, the CEEB fan-out — all were logic errors inside single functions, catchable in isolation.
Integration	How the CSV-driven pipeline stages compose (<code>build_features.build()</code> → <code>build_modeling_dataset</code> ’s freeze steps)	Catches column-contract breaks between stages that unit tests on either side, run alone, cannot see.
System	The five-script CSV pipeline run end to end against the real exported data (<code>csv_exports/</code>), and the full Docker/Postgres pipeline (<code>docker compose up</code> , <code>etl/run_all.py</code>) against a freshly started database	Confirms the whole thing actually runs, not just that each piece is individually correct; this is also where the missing <code>scikit-learn</code> dependency (see §4) was caught.

continued on next page

Table 1 continued

Type	What it covers	Why it matters here
User Acceptance (UAT)	Planned reviews of the two data dictionaries, <code>modeling_dataset.csv</code> , and the rigor/clustering/benchmarking outputs, by the project client (Bob Henkins, NU Undergraduate Admissions, and Adam) and, per the assignment’s “we may engage the other team,” a second student team	The platform’s entire purpose (per the report’s Section 1) is serving admissions officers standardized, auditable context — correctness by our own tests is necessary but not sufficient; the client has to find the output usable.

The Docker/Postgres pipeline was verified end to end on 2026-07-17 while this plan was being finalized: `docker compose up -d db` followed by `docker compose run -rm etl` completed successfully in **~106 seconds** against the raw data under `data/updated-sheng/`, and `schools_org_all`’s CEEB match count of **16,508** matched the CSV-path result exactly, cross-validating that both paths compute the same result from the same join logic. That run is now covered by an automated test, `tests/test_docker_pipeline.py`, which *skips* rather than fails when Docker isn’t reachable or the raw data isn’t present — the same pattern `test_system_pipeline.py` already used for the CSV-only path.

One gap remains, no longer undocumented but still not automatic. The reproducibility gap for a genuinely fresh clone is real, and running the pipeline once locally does not remove it. The directory `data/updated-sheng/` — which holds Sheng’s combined schools export, the client’s org export, and the raw CRDC/EDFacts data — is gitignored, and correctly so: it totals roughly **2.6 GB** (a single CRDC folder alone is 794 MB, and two EDFacts files are 938 MB and 875 MB, each individually over GitHub’s 100 MB per-file limit), far past what is reasonable to version in git. As of 2026-07-17 this is a documented manual step rather than an unwritten gap: the main `README.md` now links the team’s Google Drive folder holding the data and instructs a contributor to download it into `data/updated-sheng/` before running the Docker path. A new team member or a CI runner cloning the repo fresh must still perform that download manually — `test_docker_pipeline.py` will correctly skip until they do — but the hand-off itself is written down. Section 4 gives the full setup instructions, along with the remaining nice-to-have of a small sanitized fixture that would let this path run in CI without the full 2.6 GB.

Relatedly, `etl/combine_schools.py`’s SQL-backed join functions (`build_schools_org_enriched`, `build_schools_org_all`, `build_public_schools_enriched`, `build_private_schools_enriched`, and `build_cps_nces_crosswalk`) are now exercised end to end by `test_docker_pipeline.py` and its row-count cross-check, a real improvement over

having had zero coverage. This remains system-level assurance — that the whole run succeeded with the right row count — rather than unit-level coverage of each function’s specific join semantics, so dedicated unit tests for these functions, isolated from a live database, are still a gap.

Two areas are explicitly **not** covered by this plan, documented rather than silently skipped. Load and performance testing is out of scope, because this is a batch ETL pipeline re-run at most annually (per the report’s reproducibility goal), not a service under concurrent load. Security testing is likewise deferred: the platform has no external-facing API or authentication surface yet, and this should be revisited if one is added.

2 Test Scenarios

Unit test scenarios (by module)

`build_features.py`. The unit tests cover bucket-midpoint parsing across every observed format — exact ranges ("26% - 50%"), open-ended high ("Over 90%", "greater than 20"), open-ended low ("10% or fewer"), bare numbers ("0"), unparseable strings, and null input. Winsorization must clip extreme outliers at the 1st/99th percentile without crashing on all-null input. Sector classification is checked for a public HS, a private HS via `nu_type`, a private HS via a school-side `pss_id` record with **no** `nu_type` (the exact case a real bug missed), an org-only row with no school-side match, and the mutual exclusivity of the public and private flags. IB flag gating verifies that a **review**-tier match counts as a candidate while a **reject**-tier match does not (a real bug counted both) and that no-match rows are 0 rather than null. Finally, LEAID derivation must take `nces_id_12[:7]` and return null when `nces_id_12` is itself null, never falling back to the broken 5-character `leaaid` column.

`combine_schools.py`. Name normalization maps "Saint" → "ST" and "HS" → "HIGH SCHOOL", strips punctuation, and is null-safe. CEEB tie resolution keeps the CEEB on the best-tier match while nulling and flagging the loser; ties at equal tier prefer an exact name match; a frame with no duplicates is left untouched; and a unique CEEB is never modified.

`build_rigor_classification.py`. Z-scoring must produce mean-zero/std-one output on normal input, all-NaN output on zero-variance input (never dividing by zero), and NaN passthrough. The weighted composite must match a plain weighted average under full coverage, **reallocate weight proportionally** when a component is missing (rather than zero-imputing it or dropping the row), yield NaN (not 0) when zero components are available, and ignore a zero-weight component — IB in the default scheme — even when it is present. Tier assignment must produce five roughly-equal quintile buckets in the correct ordinal order (lowest score → “Below Average”), leaving NaN scores untiered rather than defaulting them into a tier.

`build_modeling_dataset.py`. The minimum-size freeze drops schools below the enrollment floor but **keeps** schools with unknown (null) enrollment rather than treating null as “too small,” and the boundary value is inclusive. Universe restriction leaves only public and private HS rows, dropping `other/oos`. The sentinel scrub turns negative sentinel codes in rate- and score-named columns into null while leaving a legitimate 0 untouched, and it never scrubs a non-suspect column even when that column holds a value that would be a sentinel elsewhere.

Integration test scenarios

The synthetic fixture — covering a public HS, a private HS via `pss_id`, a private HS via `nu_type`, and an org-only row — must survive `build_features.build()` →universe restriction →the minimum-size freeze with its sector labels intact and with no row collapsing into another. Chaining these steps must also produce no duplicate school identities, a stand-in for the class of bug the real CEEB fan-out was.

System test scenarios (automated, verified)

Each of the five CSV pipeline scripts — `build_features.py`, `build_modeling_dataset.py`, `build_rigor_classification.py`, `build_clustering.py`, and `build_benchmarking.py` — must run to completion against the real `csv_exports/` data and produce output of the expected shape: a row-count floor met, the required columns present, and categorical outputs constrained to their valid value sets. All five pass as of this writing. Separately, `docker compose up` must bring up Postgres 16 and the ETL container, `etl/run_all.py` must complete against it, and `schools_org_all`’s row count must then match the CSV-path result; both pass, completing in ~106 seconds. That second test runs only when Docker is reachable and the raw data is present locally, auto-skipping otherwise, since neither condition holds on a fresh clone (see the gap below and §4).

Remaining gap — not this pipeline’s correctness, but who can run it

The Docker/Postgres system test above is real and passing, but only on machines that already have `data/updated-sheng/`’s ~2.6 GB present locally (correctly gitignored, not a bug — see §4). A fresh clone, or a CI runner, still cannot run it without a manual download step first — now documented in `README.md` as a linked team Drive folder, not an unwritten gap, but still a step a human must perform rather than something automated. A small sanitized fixture of the data, scoped to just enough rows to exercise the joins, remains a nice-to-have for eventually running this test in CI without the full 2.6 GB, and is not built yet.

UAT scenarios (planned — none of these have happened yet)

Three acceptance checks are planned. First, Bob and Adam should be able to locate, in `data_dictionary_schools_org_enriched.csv` and `data_dictionary_modeling_dataset.csv`, the source, vintage, and description for any variable they ask about — literally what they requested in the 2026-07-14 meeting. Second, a reviewer unfamiliar with the pipeline should be able to open `modeling_dataset_v1_2026-07-17.csv` and correctly interpret `rigor_tier_label`, `sector`, and `funding_source` from the data dictionary alone, without reading the ETL code. Third, the “other team” named in the assignment’s UAT engagement note should be able to reproduce one full pipeline run from `csv_exports/` independently and obtain matching row counts, without anyone on our team walking them through it live.

3 Test Case Design

Full test cases live as executable code in `tests/`, not merely described here, so they cannot drift out of sync with the actual pipeline. Table 2 lists representative examples in the format the rubric asks for — steps, data input, and expected result.

Table 2: Representative test cases (steps / data input / expected result).

ID	Test	Input	Steps	Expected result
UT-01	<code>test_is_private_hs_via_pss_id_only</code>	Fixture row: <code>school_id="S2", pss_id="P1", nu_type=NaN, school_level=NaN</code>	Run <code>build(df)</code>	<code>is_private_hs == True, sector == "private"</code> — regression test for a real bug where such rows fell into "other/oos".
UT-02	<code>test_reject_tier_does_not_count</code>	Fixture row with <code>ib_school_id</code> set, <code>ib_match_tier="reject"</code>	Run <code>build(df)</code>	<code>ib_flag_candidate == 0</code> — regression test for a real bug where reject-tier matches were counted as confirmed.
UT-03	<code>test_missing_component_reallocates_weight_proportionally</code>	<code>comp = {"a": [1, NaN], "b": [3, 4], "c": [5, 6]}, weights={"a": .5, "b": .25, "c": .25}</code>	Call <code>weighted_composite(comp, weights)</code>	Row 1 score = $4 \cdot .5 + 6 \cdot .5$ (weight renormalized over available components).

continued on next page

Table 2 continued

ID	Test	Input	Steps	Expected result
UT-04	<code>test_keeps_the_best_tier_match</code>	Two schools sharing CEEB 050222: one <code>auto_accept</code> exact-name match, one <code>review</code> -tier different school	Call <code>resolve_ceeb_ties(df)</code>	<code>auto_accept</code> row keeps the CEEB; <code>review</code> row's CEEB is nulled and flagged.
IT-01	<code>test_full_chain_on_fixture</code>	4-row fixture (public HS, 2 private HS variants, 1 org-only)	<code>build()</code> → <code>restrict_to_hs_universe()</code> → <code>apply_min_size_freeze()</code>	Only public/private rows remain; each is public XOR private; the <code>pss_id</code> -only private school survived the chain.
ST-01	<code>test_build_rigor_classification</code> (system)	Real <code>modeling_dataset_v1*.csv</code> (34,392 rows)	Run <code>build_rigor_classification.py</code> via subprocess	Exit code 0; output has <code>rigor_tier_label</code> ; every non-null value is one of the five valid tier names.
ST-02	<code>test_full_pipeline_completes_successfully</code> (<code>tests/test_docker_pipeline.py</code>)	Raw data under <code>data/updated-sheng/</code> — present on this development machine, not on a fresh clone (~2.6 GB, git-ignored)	Bring up Postgres via <code>docker compose up -d db</code> , wait healthy, <code>docker compose run -rm etl</code>	Exit code 0; “Pipeline completed successfully” in output. Auto-skips if Docker or the raw data isn't present, so it's automated where its precondition holds rather than manual everywhere.
ST-03	<code>test_schools_org_all_row_count_matches_csv_path</code> (<code>tests/test_docker_pipeline.py</code>)	Same as ST-02	Query <code>schools_org_all</code> row count from the live database after ST-02	Matches the CSV-path row count (53,966) — cross-validates both paths independently.

Adding a test case follows one convention: any new bug found gets a regression test named for the bug, not just for the function it lived in. This matches the pattern already used throughout, described directly in `test_build_features.py`'s docstring.

4 Test Environment Setup

Two environments are supported, matching how the pipeline is actually run day to day. The first, **CSV-only (no database)**, is what most contributors use: it requires Python 3.11+ (matching `etl/Dockerfile`'s `python:3.11-slim`; developed and tested here on 3.13.5) to-

gether with the packages pinned in `etl/requirements.txt`, and it operates directly on `csv_exports/*.csv` — this is what the system tests of §1 run against. The second, the **full Docker stack**, is defined by `docker-compose.yml`, which brings up Postgres 16 (`capstone-db`, port 5433→5432) and an ETL container (`capstone-etl`, built from `etl/Dockerfile`) that runs `etl/run_all.py` against it; it is needed for `combine_schools.py`’s SQL-backed builds and for anyone re-deriving `csv_exports/` from raw source data rather than reading the already-exported CSVs.

The Docker stack was verified working end to end on 2026-07-17: `docker compose up -d db`, then `docker compose run -rm etl`, completed successfully in ~106 seconds and produced a `schools_org_all` row count matching the CSV path exactly. That is covered by the automated test `tests/test_docker_pipeline.py`.

As of 2026-07-17 the data hand-off is documented, not merely recommended. The directory `data/updated-sheng/` — Sheng’s combined schools export, Bob’s org export, and the raw CRDC/EDFacts assessment data — is gitignored, and correctly so: it is roughly **2.6 GB** (CRDC data alone is 794 MB, and two EDFacts files are 938 MB and 875 MB, each individually over GitHub’s 100 MB per-file limit), far past what is reasonable to version in git. That limit was checked directly against GitHub’s 100 MB hard per-file cap before attempting anything, rather than discovered through a failed push. The data is instead distributed through a team Google Drive folder linked from the main `README.md`’s Quickstart section (request access from Max if it is not visible). A new contributor or CI runner still needs to download it before the Docker/Postgres path or `tests/test_docker_pipeline.py` will do anything beyond auto-skip, but that is now a documented three-minute setup step rather than an undocumented gap. A small sanitized fixture, scoped to just enough rows to exercise the joins, remains a nice-to-have for eventually running this path in CI without the full 2.6 GB, but is not built yet.

Test-specific dependencies are kept in `requirements-test.txt` (pytest 9.1.1), separate from `etl/requirements.txt`, since pytest is a development- and test-time dependency rather than a pipeline runtime one. Writing this plan already caught one real environment gap: `etl/requirements.txt` was missing `scikit-learn`, which `build_clustering.py` requires, meaning the documented Docker environment could not actually run that script. It was fixed as part of this work (see `etl/requirements.txt`) and confirmed by the successful Docker build above, not merely asserted.

Configuration is read by `etl/config.py` from the environment variables `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASS` (defaults `localhost:5432/capstone/postgres`), overridden by `docker-compose.yml`’s `capstone/capstone/capstone` when run in Docker, while `NU_MASTER_PATH` and `SCHOOLS_CEEB_PATH` point at the raw data files under `data/`. The full suite is run as follows:

```
pip install -r requirements-test.txt -r etl/requirements.txt
```

```
pytest tests/ -v
```

The system tests (`test_system_pipeline.py` and `test_docker_pipeline.py`) auto-skip when their respective preconditions — exported CSVs, or Docker together with the raw data — are not present, so the suite still runs the unit and integration tests on a checkout without either.

5 Test Execution Plan

Execution is tied to the project’s own weekly schedule rather than an invented cadence, and Table 3 is honest about what is and is not formally assigned. No RACI document exists in this repo: `docs/BOB_BRIEFING.md` references “per the RACI, master database & data pulls is Max/Qifan’s workstream,” which is the closest documented scope to this pipeline’s code, but no RACI covering *testing* specifically has been written. Where a name appears below it is inferred from that one existing reference, not from a formal assignment, and is flagged as its own action item rather than silently assumed.

Table 3: Test execution phases, timing, and (candidate) ownership.

Phase	When	What runs	Who
Unit + integration, every commit	Ongoing, starting now (Week 4)	<code>pytest tests/test_-build_features.py tests/test_combine_schools.py tests/test_build_rigor_classification.py tests/test_build_modeling_dataset.py tests/test_integration_pipeline.py</code>	Whoever touches <code>etl/*.py</code> — run locally before pushing; no CI is wired up yet (see Defect Management), so this is currently a manual discipline, not an enforced gate.
System test, before every new versioned dataset	Whenever <code>modeling_-dataset_v<N></code> is cut	<code>pytest tests/test_system_pipeline.py</code>	Whoever cuts the version.

continued on next page

Table 3 continued

Phase	When	What runs	Who
Full Docker system test	Before any DB-schema-affecting change ships (new table, changed join); verified passing on 2026-07-17	<code>pytest tests/test_-docker_pipeline.py</code> (<code>docker compose up, etl/run_all.py</code> , then a row-count cross-check) — auto-skips on machines without Docker/the raw data (see §4)	Closest existing RACI scope: Max/Qifan (“master database & data pulls” per <code>BOB_-BRIEFING.md</code>) — needs explicit confirmation, not assumed.
Hyperparameter/validation-specific testing	Week 7 (per the project’s own schedule — Section 4.6 of the report)	Train/test/validation split checks, overfitting/underfitting checks — not yet built; this plan’s system/integration tests are the pre-requisite groundwork for that phase	Not assigned — no RACI line covers modeling/validation specifically; needs a decision, not a default.
UAT round 1	As soon as Bob/Adam have bandwidth (not gated on a specific week)	Structured review of both data dictionaries + <code>modeling_-dataset.csv</code> , using the UAT scenarios in §2	Bob, Adam.
UAT round 2 (peer team)	Before Week 9 presentation prep	The engaged “other team” independently reproduces one pipeline run and reviews the rigor/clustering/benchmarking docs for clarity	Second student team + our team.

Several dependencies constrain this schedule. The system tests depend on `csv_exports/` being current, so it is regenerated via the chain in §1 before they run. UAT round 2 depends on round 1 feedback having been incorporated first, so the peer team is not reviewing something already known to be wrong. The Docker system test runs today on any machine with Docker and the raw data present; a fresh clone or CI runner needs the manual download step documented in `README.md` first (see §4), and stays unavailable until a sanitized fixture removes that manual step entirely. Writing this plan also surfaced a standing action item: a real RACI covering test ownership specifically does not exist yet, and the team should produce one — even a short one — rather than leaving this plan to infer ownership from a

single line in `BOB_BRIEFING.md`.

6 Defect Management

No formal issue tracker is wired up yet, so this section states the process going forward rather than one already running.

Defects found through testing, whether automated or via manual and UAT review, are filed as GitHub Issues on `mchalekson/498-capstone`, tagged by the pipeline stage affected (`etl`, `rigor-classification`, `clustering`, `benchmarking`, or `data-quality`). Each issue states what broke, the failing test if the defect is automated or the specific input that triggered it otherwise, and the expected versus actual output — the same information already present in this session’s fix commits, of which the CEEB fan-out fix is an example, documenting the exact failing pattern with real school names and CEEB values.

Severity is assigned on three tiers. A *blocking* defect breaks a pipeline stage outright, as the missing `scikit-learn` dependency did, and is fixed before the next system-test run. A *data-quality* defect produces output that is wrong — the CEEB fan-out and the IB sector-classification bug are examples — and is fixed before the next versioned `modeling_dataset` cut, and documented in the relevant `docs/*.md` memo regardless of when the fix lands. A *documentation* defect is a case where code and docs disagree but the code itself is correct, such as the `parse_bucket_midpoint` docstring typo found while writing this plan’s tests; it is low priority and batched into the next doc-touching commit.

Issue status — open, in progress, or resolved — lives on the GitHub Issue itself, with no separate spreadsheet, to avoid the two-sources-of-truth problem this project has already flagged for data-vintage tracking. Every defect fix ships with a regression test named for the bug (per the convention in §3), so a defect is never merely “resolved,” it is resolved-and-covered.

7 Regression Testing

The full unit-and-integration suite (`pytest tests/ -k "not system"`) runs before every commit that touches `etl/*.py`, rather than on a fixed calendar cadence — appropriate for a small team iterating quickly rather than a large one needing a scheduled batch window. Every previously found bug in this pipeline has a corresponding test — the sector-classification bug, the IB gating bug, the LEAID derivation bug, the CEEB fan-out, and the docstring/behavior mismatch in `parse_bucket_midpoint` — so regression here specifically means “did a fixed bug come back,” not just “did anything change.”

At the full-pipeline level, the system-test suite reruns the entire five-script CSV chain against the current `csv_exports/` data whenever any script in that chain changes, catching

regressions that only manifest at real production scale, where small-fixture unit tests can miss scale-dependent issues such as the quintile-tier bucketing needing a large-enough population. On machines with Docker and the raw data available, `test_docker_pipeline.py` extends this to the database-backed join functions as well, cross-checking the DB path’s row counts against the CSV path’s — something not yet possible in CI (see §4). That absence of CI is a real gap rather than one glossed over: the recommendation is to wire up GitHub Actions to run the non-system tests on every push once the team has bandwidth, so regression testing stops depending on individual discipline.

8 Documentation and Reporting

Test results are reported the same way every other finding in this project has been — as dated updates in the relevant `docs/*.md` file (`EDA_features_joined.md`, `RIGOR_CLASSIFICATION.md`, `CLUSTERING.md`, `BENCHMARKING.md`), not as a separate test-report artifact disconnected from the data findings it relates to. This test plan itself (`docs/TEST_PLAN.md`) is the canonical reference for test *process*, while individual *results* stay attached to the deliverable they validate.

Reporting frequency follows the work: results are documented after every system-test run that precedes a new versioned dataset, and immediately whenever a defect is found rather than in batches, so that (per the severity tiers above) blocking and data-quality defects are written into the relevant memo right away.

The client-facing summary channel is `docs/BOB_BRIEFING.md`, already the established pattern for this — it has flagged Bob-relevant items such as the CEEB corruption finding throughout the project — so test and defect findings that affect data Bob or Adam would review are added there specifically, not only to the internal EDA memos. UAT feedback from Bob, Adam, and the peer team is logged in the same `BOB_BRIEFING.md` update sections, matching its existing “what we resolved ourselves” and “what we still need from you” structure, so that acceptance feedback and technical defect tracking do not fragment into separate, hard-to-reconcile systems.

9 Sample Test Report

Per the assignment instructions (“test reports... with sample/test data, inputs, and expected outputs”), the run below is real rather than illustrative — captured 2026-07-17 against this project’s actual code and data, not a mocked-up example.

Automated suite — full run

```
$ pytest tests/ -v
...
57 passed in 115.31s (0:01:55)
```

All 57 tests across all six test files passed: `test_build_features.py` (21), `test_build_modeling_dataset.py` (7), `test_build_rigor_classification.py` (11), `test_combine_schools.py` (9), `test_docker_pipeline.py` (2), `test_integration_pipeline.py` (2), and `test_system_pipeline.py` (5). This run included the Docker-gated tests, meaning the raw source data was present and the full Postgres pipeline was exercised rather than skipped.

Sample test case, executed — UT-01 (private-school sector classification)

Field	Value
Test data / input	A fixture row: <code>school_id="S2"</code> , <code>pss_id="P1"</code> , <code>nu_type=NaN</code> , <code>school_level=NaN</code> (see <code>tests/conftest.py</code>).
Action	<code>build(tiny_schools_org_all)</code> .
Expected output	<code>is_private_hs == True</code> , <code>sector == "private"</code> .
Actual output	<code>is_private_hs == True</code> , <code>sector == "private"</code> .
Result	PASS.

Sample test case, executed — ST-02/ST-03 (Docker/Postgres system test)

Sample query against the live database — real data, not a mock

To demonstrate that the database is genuinely queryable end to end, two arbitrary example queries were run against `schools_org_all` (53,966 rows) in the running Postgres container. The first returns the schools with the highest AP-test-taking intensity:

```
SELECT school_name, state, nu_avg_num_ap_tests_taken, nu_avg_freshman_sat
FROM schools_org_all
WHERE nu_avg_num_ap_tests_taken IS NOT NULL
ORDER BY nu_avg_num_ap_tests_taken DESC
LIMIT 5;
```

The second aggregates AP course counts and graduation rates by state, restricted to public high schools in states with 500 or more schools:

Field	Value
Test data / input	Raw source data under <code>data/updated-sheng/</code> (~2.6 GB — see §4); a freshly created Postgres 16 container.
Action	<code>docker compose up -d db</code> , wait healthy, <code>docker compose build etl</code> , <code>docker compose run -rm etl</code> , then query <code>schools_org_all</code> row count.
Expected output	Exit code 0; “Pipeline completed successfully” in output; <code>schools_org_all</code> row count matches the CSV-path result (53,966).
Actual output	Exit code 0; pipeline completed in ~106 seconds; <code>schools_org_all</code> row count = 53,966, exactly matching the CSV path.
Result	PASS.

School	State	Avg AP tests/student	Avg freshman SAT
School of Science and Engineering	TX	15.49	1290
School for the Talented and Gifted	TX	14.16	1250
BASIS Chandler	AZ	12.33	1260
BASIS Peoria	AZ	11.76	1200
IDEA College Preparatory Pharr	TX	11.58	1080

```
SELECT state, COUNT(*) AS n_schools,
       ROUND(AVG(crdc_ap_courses)::numeric, 1) AS avg_ap_courses_offered,
       ROUND(AVG(grad_rate_2021)::numeric, 1) AS avg_grad_rate
FROM schools_org_all
WHERE school_level = 'High'
GROUP BY state HAVING COUNT(*) > 500
ORDER BY avg_grad_rate DESC LIMIT 5;
```

State	# Schools	Avg AP courses offered	Avg grad rate
TX	1,959	12.5	88.8%
PA	716	11.0	88.5%
WI	560	10.0	87.9%
MO	658	9.8	87.6%
NY	1,182	10.9	86.5%

Worth noting honestly rather than cropping out of the sample: this same query returns a null average grad rate for Washington state (657 schools). That is not a bug but a real coverage gap already documented in the data dictionary, since EDFacts grad-rate data is not complete for every state. Including it here is deliberate, because a sample test report that shows only clean rows is not an honest sample.